

Tema 8 - Lenguajes Intermedios

Profesor Ramón López-Cózar Delgado, Curso 21-22

Lenguaje Intermedio:

- Es una representación más abstracta y uniforme que un lenguaje máquina concreto.
- Su misión es descomponer las expresiones complejas en binarias y las sentencias complejas en sentencias simples.
- **Ventajas:**
 - Permite una fase de análisis semántico independiente de la máquina.
 - Se pueden realizar optimizaciones sobre el código intermedio, las complejas rutinas de optimización son independientes de la máquina.
- **Desventajas:**
 - Introduce en el compilador una nueva fase de traducción.

Tipos

- Árbol Sintáctico
- Árbol Sintáctico Abstracto
 - Todos los nodos del árbol representan símbolos terminales.
 - Los nodos hijos son operandos y los nodos internos son operadores.
- Grafo Dirigido Acíclico (GDA)
- Notación posfija
 - Usado en máquinas abstractas
- N-Tuplas
 - Cada sentencia del lenguaje intermedio consta de N elementos:
 - $(Operador, Operando_0, Operando_1, \dots, Operando_{N-1})$
 - Los más usuales son los tercetos (tripletas) y los cuartetos (cuádruplas), llamados también código de tres direcciones.
 - Se le llama así porque cada operación tiene a lo sumo 3 operandos, aunque existen excepciones con menos.

Tercetos:

`<operador>, <operando_1>, <operando_2>`

Ejemplo: `d = a + b * c`

```
[1] (*, b, c)
[2] (+, a, [1])
[3] (=, d, [2])
```

Cuartetos:

`<operador>, <operando_1>, <operando_2>, <resultado>`

Ejemplo: `d = a + b * c`

```
(*, b, c, temp1)
(+, a, temp1, temp2)
(=, temp2, --, d)
```

Lenguaje Intermedio de Cuartetos:

Proposiciones

- **if-then-else:** $S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$

```

s_comienzo_1:
    código de asignación de expresión E
    if E goto s_then_1
    goto s_else_1

s_then_1:
    código de sentencias de S1
    goto s_despues_1

s_else_1:
    código de sentencias de S2

s_despues_1:

```

- **Switch-Case**

```

• Case: switch E
    begin
        case V1 : S1 ;
        case V2 : S2 ;
        ...
        case Vn-1 : Sn-1 ;
        default : Sn
    end;

```

```

s_inicio_case:
    código evaluación de E en t
    if t ≠ V1 goto L1
    código S1
    goto s_despues_1
L1:
    if t ≠ V2 goto L2
    código S2
    goto s_despues_1
    ...
Ln-2:
    if t ≠ Vn-1 goto Ln-1
    código Sn-1
    goto s_despues_1
Ln-1:
    código Sn
s_despues_1:

```

Generación de código en cuartetos

- Cuando se genera código de tres direcciones, se construyen nombres temporales para los nodos inferiores del árbol sintáctico, inicialmente se creará un nuevo nombre cada vez que se necesite una variable temporal.
- Los atributos sintetizados, entre otros son:
 - *lugar*: Contendrá el valor del atributo.
 - *codigo*: Contendrá la secuencia de proposiciones en tres direcciones.
- La función *tempnuevo* devuelve una secuencia de nombres distintos $t_1, t_2, \dots, t_n$ en respuesta a sucesivas llamadas.
- La función *etiqnueva* devuelve una secuencia de nombres distintos $e_1, e_2, \dots, e_n$ en respuesta a sucesivas llamadas.
- Por notación, se utiliza *gen*(...) para representar una proposición de tres direcciones concreta dada como conjunto de argumentos.
- **Expresiones Aritméticas y Asignación**

$S : id := E$	$\{ S.codigo = E.codigo \parallel gen(id.lugar := E.lugar); \}$
$E : E_1 + E_2$	$\{ E.lugar = tempnuevo ;$ $E.codigo = E_1.codigo \parallel E_2.codigo \parallel gen(E.lugar := E_1.lugar + E_2.lugar); \}$
$E : E_1 * E_2$	$\{ E.lugar = tempnuevo ;$ $E.codigo = E_1.codigo \parallel E_2.codigo \parallel gen(E.lugar := E_1.lugar * E_2.lugar); \}$
$E : - E_1$	$\{ E.lugar = tempnuevo ;$ $E.codigo = E_1.codigo \parallel gen(E.lugar := - E_1.lugar); \}$
$E : (E_1)$	$\{ E.lugar = E_1.lugar ;$ $E.codigo = E_1.codigo ; \}$
$E : id$	$\{ E.lugar = id.lugar ;$ $E.codigo = ' ' ; \}$

- **Expresiones Lógicas**

- Existen dos métodos para evaluar las expresiones lógicas
 - **Mediante la representación numérica:** Asociar valores numéricos a true y false; por ejemplo en C true es cualquier valor diferente de 0, y false es 0.
 - **Mediante el flujo de control:** Representar una expresión lógica dependiendo de la posición alcanzada en el programa.

Representación numérica

$E : E_1 \text{ or } E_2$	$\{ E.lugar = tempnuevo ;$ $gen(E.lugar := E_1.lugar \text{ or } E_2.lugar); \}$
$E : E_1 \text{ and } E_2$	$\{ E.lugar = tempnuevo ;$ $gen(E.lugar := E_1.lugar \text{ and } E_2.lugar); \}$
$E : \text{not } E_1$	$\{ E.lugar = tempnuevo ;$ $gen(E.lugar := \text{not } E_1.lugar); \}$
$E : (E_1)$	$\{ E.lugar = E_1.lugar ; \}$
$E : id \text{ oprel } id$	$\{ E.lugar = tempnuevo ;$ $gen('if' id_1.lugar \text{ oprel.op } id_2.lugar \text{ goto sigt prop+3});$ $gen(E.lugar = 0);$ $gen(\text{goto sigt prop+2});$ $gen(E.lugar := 1); \}$
$E : \text{true}$	$\{ E.lugar = tempnuevo ;$ $gen(E.lugar := 1); \}$
$E : \text{false}$	$\{ E.lugar = tempnuevo ;$ $gen(E.lugar := 0); \}$

A and not B or true	A > B or C = D
temp1 := not B	0: if A > B goto 3
temp2 := A and temp1	1: temp1 := false
temp3 := true	2: goto 4
temp4 := temp2 or temp3	3: temp1 := true
	4: if C = D goto 7
	5: temp2 := false
	6: goto 8
	7: temp2 := true
	8: temp3 := temp1 or temp2

Flujo de control

$E : E_1 \text{ or } E_2$	<pre> { E₁.verdadera = E.verdadera ; E₁.falsa = etiqnueva ; E₂.verdadera = E.verdadera ; E₂.falsa = E.falsa ; E.codigo = E₁.codigo gen (E₁.falsa ':') E₂.codigo ; }</pre>
$E : E_1 \text{ and } E_2$	<pre> { E₁.verdadera = etiqnueva ; E₁.falsa = E.falsa ; E₂.verdadera = E.verdadera ; E₂.falsa = E.falsa ; E.codigo = E₁.codigo gen (E₁.verdadera ':') E₂.codigo ; }</pre>
$E : \text{not } E_1$	<pre> { E₁.verdadera = E.falsa ; E₁.falsa = E.verdadera ; E.codigo = E₁.codigo ; }</pre>
$E : (E_1)$	<pre> { E₁.verdadera = E.verdadera ; E₁.falsa = E.falsa ; E.codigo = E₁.codigo ; }</pre>
$E : \text{id oprel id}$	<pre> { E.codigo = gen ('if' id₁.lugar oprel.op id₂.lugar 'goto' E.verdadera) gen ('goto' E.falsa) ; }</pre>
$E : \text{true}$	<pre> { E.codigo = gen ('goto' E.verdadera) ; }</pre>
$E : \text{false}$	<pre> { E.codigo = gen ('goto' E.falsa) ; }</pre>

$E := A > B \text{ or } C = D$

```

    if A > B goto Everdadera
    goto L1
L1:  if C = D goto Everdadera
    goto Efalsa
Everdadera: E = verdadera
    goto L2
Efalsa: E = falsa
L2:
```

- Condicional if-then

S : if E then S ₁	<pre> { E.verdadera = etiqnueva ; E.falsa = S.siguiente ; S₁.siguiente = S.siguiente ; S.codigo = E.codigo gen ('if' !E.lugar goto E.falsa) gen (E.verdadera ':') S₁.codigo gen ('goto' S.siguiente) ; } </pre>
------------------------------	---

comienzo:	código de asignación de expresión E
	if !E.lugar goto E.falsa
etiqnueva:	código de sentencias de S ₁
	goto S.siguiente

- Condicional if-then-else

S : if E then S ₁ else S ₂	<pre> { E.verdadera = etiqnueva1 ; E.falsa = etiqnueva2 ; S₁.siguiente = S.siguiente ; S₂.siguiente = S.siguiente ; S.codigo = E.codigo gen ('if' !E.lugar goto E.falsa) gen (E.verdadera ':') S₁.codigo gen ('goto' S.siguiente) ; gen (E.falsa ':') S₂.codigo ; } </pre>
--	--

comienzo:	código de asignación de expresión E
	if !E.lugar goto E.falsa
etiqnueva1:	código de sentencias de S ₁
	goto S.siguiente
etiqnueva2:	código de sentencias de S ₂

- Bucle while

S : while E do S ₁	<pre> { S.comienzo = etiqnueva1 ; S.despues = etiqnueva2 ; S.codigo = gen (S.comienzo ':') E.codigo gen ('if' E.lugar != 0 goto S.despues) S₁.codigo gen ('goto' S.comienzo) gen (S.despues ':') ; } </pre>
-------------------------------	---

etiqnueva1:	código de asignación de expresión E
	if E.lugar = 0 goto etiqnueva2
	código de sentencias de S ₁
	goto etiqnueva1
etiqnueva2:	

Ejemplo de Lenguaje Intermedio de Máquina Abstracta: Código P

- El código P (P-Code) comenzó como un código ensamblador objeto estándar producido por varios compiladores Pascal en la década de los 70 y principios de los 80.

- Diseñado para ser el código real de una máquina abstracta denominada máquina P para la cual se escribieron intérpretes para varias arquitecturas.
- La idea general era que los compiladores de Pascal se transportaran fácilmente a diferentes arquitecturas, se trata de un código más cercano al código máquina que al de tres direcciones.
- La máquina se compone de:
 - Conjunto de instrucciones.
 - Conjunto de datos.
 - Memoria organizada mediante pila y montículo (heap)
 - Registros de Control de la Memoria